Differential Analysis of LLMs in Text2SQL Generation

Jaykumar Patel
patel.jay4802@utexas.edu

An-Jung Huang
ajhuang@utexas.edu

Ryan Kolm
ryankolm@utexas.edu

Abstract

Large Language Models (LLMs) have shown remarkable capabilities in translating natural language questions into executable SQL queries, a task known as Text2SQL. In this work, we conduct a systematic evaluation of several state-of-the-art LLMs and two distinct Text2SQL frameworks: agentless and agent-based pipelines. Our experiments assess accuracy, consistency, and execution robustness on the BIRD-SQL Dev Set. By enabling users to query structured databases through natural language, effective Text2SQL systems have the potential to democratize data access, allowing non-developers to retrieve complex information from structured databases without needing specialized technical skills. Our study aims to inform future design choices for building more reliable and accessible natural language interfaces to databases.

1 Introduction

As large language models (LLMs) continue to improve, an interesting application of LLMs is in the generation of SQL queries from natural language. This is known as Text2SQL. Text2SQL can enable non-experts to extract valuable information from a database without having to learn complex query languages like SQL. By simply describing the data they want in plain English, users can leverage LLMs to translate their requests into SQL queries that interact directly with a database. This has the potential to democratize data access across organizations, reduce reliance on developers, and improve overall efficiency in data-driven decision-making.

In this project, we compare the performance of different LLMs in generating SQL queries from natural language. We explore 2 different frameworks, agentless and agent-based, and 3 different LLMs, Claude 3.5 Sonnet, GPT-4o mini, and Llama 3.1 8B. In the agentless framework, the descriptions of all tables in the database (relevant and irrelevant) are passed as context to the LLM when generting the SQL query given the natural language query (NLQ). In the agent-based framework, there are 3 agents that handle specific tasks such as determining the number of relevant tables, retrieving the descriptions of the relevant tables, and generating a SQL query given a natural language query. We run our agentless and agent-based frameworks with the 3 different LLMs on the BIRD-SQL Dev Set benchmark.

By systematically analyzing various LLMs and frameworks, we aim to understand how model choice and pipeline design impact the accuracy, consistency, and execution success of generated SQL queries. Our findings can help guide the development of more reliable Text2SQL systems, making database interaction easier and more intuitive.

2 Previous Work

Text2SQL has been an active area of research. Early methods primarily relied on rule-based systems [4] and semantic parsing [7], which required significant domain-specific engineering. With the rise of deep learning, encoder-decoder architectures and pre-trained language models began to dominate, enabling better conversion from natural language to SQL queries.

Recent works have explored building Text2SQL pipelines that leverage LLMs to generate SQL queries through multi-step processes. CHASE-SQL [6] and XiYan-SQL [2] both implement multi-agent frameworks organized into three key phases: retrieving relevant schema information, generating candidate queries, and selecting the most appropriate SQL statement—each of which may involve one or more specialized agents. CHASE-SQL employs methods such as divide-and-conquer chain-of-thought (CoT) reasoning, query-plan-based CoT reasoning, and online synthetic example generation to enhance candidate SQL query generation. XiYan-SQL integrates supervised fine-tuning of multiple generators and in-context learning (ICL) to produce high-quality and diverse SQL candidates. CHASE-SQL and XiYan-SQL rank 2nd and 4th respectively for the BIRD benchmark.

While these approaches demonstrate the effectiveness of multi-agent pipelines for improving Text2SQL performance, there is value in systematically comparing different LLMs and framework designs under a constant experimental setting. In our work, we conduct an evaluation of both agentless and agent-based pipelines across multiple LLMs, aiming to better understand how model choice and pipeline structure influence SQL generation quality on the BIRD-SQL Dev Set. Specifically, we evaluate not only a larger model such as Claude 3.5 Sonnet, but also smaller, potentially more economical models like GPT-4o Mini and Llama 3.1 8B. For feasibility, we employ a simplified version of the agent-based framework, utilizing a single generator rather than multiple specialized generators. Additionally, to assess the consistency of each model and framework combination, we test each NLQ on each configuration three times.

3 Methodology

In this section, we outline the methodology used for our differential analysis of LLMs in Text2SQL generation.

3.1 Model Selection

For this study, we selected the three Large Language Models (LLMs) Claude 3.5 Sonnet, GPT-4o mini, and Llama 3.1 8B. These models are widely adopted and are diverse in model size and architecture. Particularily, Claude 3.5 Sonnet has 175 billion parameters; whereas, GPT-4o mini and Llama 3.1 8B have 8 billion parameters [1]. Additionally, Llama 3.1 8B is open-source, while GPT-4o Mini and Claude 3.5 Sonnet are closed-source. By comparing these models, we can evaluate how performance varies not only with model size but also between open-source and closed-source LLMs.

Temperature controls the degree of randomness in the text generated by LLMs during inference [5]. The default temperature for Claude 3.5 Sonnet is 1.0, which represents the maximum level of randomness. For GPT-4o mini, the default temperature is 0.7; however, we increase it to 1.0 to enable greater diversity in the SQL queries generated across three runs, with the hope of improving the chances of obtaining a correct result. For Llama 3.1 8B, we keep the default lower temperature, as smaller models are generally more prone to hallucinations when exposed to higher levels of randomness.

3.2 Framework Selection

There are many ways that Text2SQL can be implemented. In this study, we compare the performance difference between 2 frameworks, one that performs without AI agents (agentless) and one that utilizes a series of AI agents (agent-based).

1. **Agentless**: In this framework, an LLM is used to generate the response directly. The NLQ and all table descriptions are provided to the **LLM**. The LLM then generates a SQL query. The agentless pipeline can be seen in Figure 1.

2. **Agent-based**: In this framework, there are distinct agents that each play a different role in the generation of the response. Before this pipeline can be used, all table descriptions must be vectorized and stored in **ChromaDB**. There are three agents: the **Relevance Agent**, **Retrieval Agent**, and **Query Agent**. The NLQ and all table descriptions are provided to the **Relevance Agent**, which determines the number of tables that are relevant, denoted as K. Then, the **Retrieval Agent** retrieves the K table descriptions from ChromaDB that are most similar to the NLQ. This process is similar to the retrieval step in a RAG system and allows us to retrieve the relevant table descriptions. Finally, the NLQ and the retrieved relevant table descriptions are provided to the **Query Agent**, which then generates the SQL query. The agent-based pipeline can be seen in Figure 2.

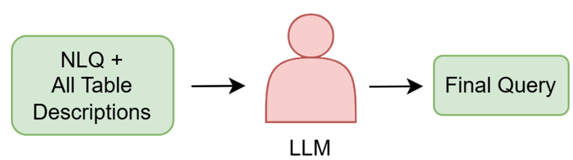


Figure 1: Agentless Text2SQL Pipeline

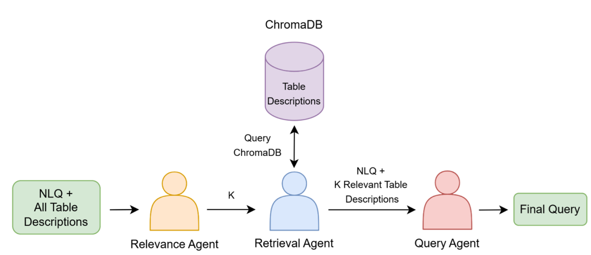


Figure 2: Agent-based Text2SQL Pipeline

The main difference between the agentless and agent-based frameworks is in the SQL generation phase of the pipeline (these are the "red" agents in the architecture diagrams): for agentless, the LLM uses the descriptions of all tables, whereas for agent-based, the query agent uses only the descriptions of relevant tables. Comparing these two frameworks provides insight into the trade-offs between simplicity and modularity. Specifically, we aim to explore whether introducing specialized agents leads to more accurate query generation. This evaluation can help guide the future development of LLM-powered database interfaces.

Our agent-based pipeline design is also inspired by leading multi-agent Text2SQL frameworks, such as CHASE-SQL and XiYan-SQL. These two approaches segment the generation process into three main phases: (1) a schema or value retrieval phase to fetch database context, (2) a candidate generation phase to produce potential SQL queries, and (3) a candidate selection phase to choose the best query among multiple options. Our pipeline incorporates the first two phases: we employ a Relevance Agent and a Retrieval Agent to retrieve the relevant table descriptions, followed by a Query Agent to generate the final SQL query. However, because we generate only a single candidate query per NLQ, a selection phase is not necessary, as there are no multiple candidates to choose from.

An important motivation for exploring agentless and agent-based frameworks is the "lost in the middle" effect [3], where LLMs struggle to effectively utilize relevant information when it is in the middle of large contexts. As the context grows—particularly with many irrelevant table descriptions—LLMs can become less accurate in their reasoning if the relevant information gets "lost" in the context. The agent-based framework addresses this by using dedicated agents (Relevance and Retrieval) to retrieve and focus only on the most relevant table descriptions. This reduces the context size and mitigates the "Lost in the middle" effect, with the hope that it can improve the model's ability to generate accurate SQL queries.

3.3 Benchmark Dataset Selection

We use BIRD-SQL as the benchmark for evaluation. It is a widely adopted benchmark in the Text2SQL community, known for its diverse databases, varying query complexity, and difficulty categorizations (simple, moderate, and challenging). As a result, BIRD-SQL allows for a fair comparison across the different models and frameworks.

The BIRD-SQL includes table descriptions for every database, a collection of NLQs, their corresponding ground-truth SQL queries, and their difficulty levels. For our experiments, we focus on the "student-club" database. This database has 8 tables making the structure complex enough for the problems to be interesting. We sample 30 questions (18 simple, 9 moderate, and 3 challenging) which we evaluate out LLMs and frameworks against.

3.4 Evaluation Metrics

We evaluate the performance of each Model+Framework combination across three main metrics: **accuracy**, **consistency**, and **execution error rate**.

- **Accuracy:** Accuracy reflects the correctness of the generated SQL queries. Rather than directly comparing the generated SQL query to the ground-truth SQL query provided by BIRD-SQL—since multiple SQL formulations can yield the same result—we assess correctness based on execution results. Specifically, we execute the generated SQL query and the corresponding ground-truth SQL query from BIRD-SQL on the database, and determine correctness by comparing their execution results. A generated query is considered correct if its execution result exactly matches the execution result of the ground-truth query. We categorize correctness based on the number of successful generations out of three attempts for each query: 0/3, 1/3, 2/3, or 3/3 correct generations.

- **Consistency:** We run each NLQ on each Model+Framework combination 3 times. A Model+Framework combination that achieves 3/3 correct generations for a query is considered highly consistent, while fluctuating results (e.g., 1/3 or 2/3 correct) indicate lower consistency.

- **Execution Error Rate:** This is the percentage of generated SQL queries that fail to execute, typically due to syntax errors, invalid table references, or improper SQL structure. A lower execution error rate reflects greater robustness and syntactic reliability. A higher execution error rate may indicate hallucinations, misunderstandings of the database schema, or general instability in query generation.

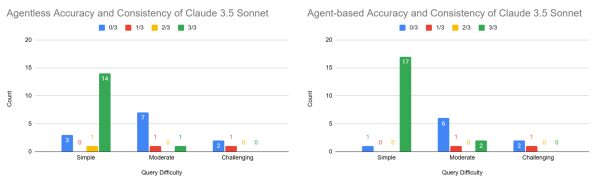
Additionally, we compare the overall performance between the agentless and agent-based frameworks to explore the trade-offs between simplicity (agentless) and modular specialization (agent-based). Each Model+Framework combination is tested on the 30 sampled queries, with three runs per query to capture consistency.

4 Results

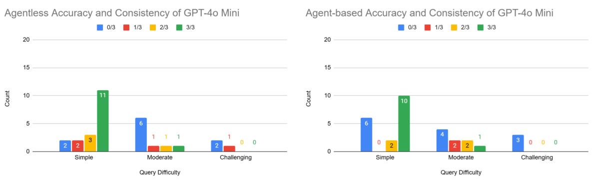
4.1 Accuracy and Consistency

We evaluate the accuracy and consistency of each Model+Framework combination by generating a SQL query for each NLQ 3 times. Then we categorize the results into 4 buckets: 0/3, 1/3, 2/3, and 3/3 correct generations. The performance for each model under agentless and agent-based frameworks is summarized in Figure 3.

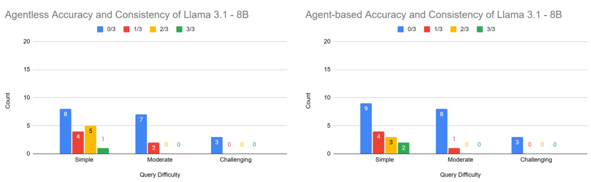
Figure 3a shows the accuracy and consistency results for Claude 3.5 Sonnet. Under the agent-based framework, Claude 3.5 Sonnet achieved near-perfect consistency on simple queries, with 17 out of 18 queries answered correctly



(a) Accuracy and Consistency of Claude 3.5 Sonnet



(b) Accuracy and Consistency of GPT-4o mini



(c) Accuracy and Consistency of Llama 3.1 8B

Figure 3: Accuracy and Consistency

3/3 times. However, for moderate queries, there was a significant drop in performance: 6 queries fell into the 0/3 category. Performance on challenging queries was worse, with only a 1 query in the 1/3 category. In the agentless framework, performance remained strong but slightly lower than agent-based for simple queries, with 14 queries answered perfectly (3/3). Moderate and challenging queries showed a similar trend of decreased accuracy and consistency. Overall, Claude 3.5 Sonnet demonstrated the highest robustness among all models, especially on simple queries, with the agent-based framework offering a slight advantage.

Figure 3b shows the accuracy and consistency results for GPT-4o Mini. Unlike Claude, GPT-4o Mini showed more variance even on simple queries. Under the agent-based framework, only 10 out of 18 simple queries were answered perfectly (3/3), with 6 queries failing completely (0/3). Moderate queries exhibited mixed success, while challenging queries were unsuccessful. In the agentless framework, GPT-4o Mini's performance on simple queries slightly improved, with 11 queries answered perfectly. However, its performance on moderate queries worsened. Interestingly, its performance on challenging queries improved slightly where 1 query was answered correctly 1/3 times. Compared to Claude, GPT-4o Mini was less consistent across runs, especially for more complex queries.

Figure 3c shows the accuracy and consistency results for Llama 3.1 8B, which was the least consistent of the three models. Under both frameworks, majority of queries, including simple ones, fell into the 0/3 correctness category. Only a small number of queries achieved perfect correctness, mainly for simple queries. Moderate and challenging queries were problematic, with most attempts failing across all runs. These results highlight the limitations of smaller or less refined models in SQL generation tasks.

Across all models, simple queries were solved more consistently than moderate and challenging ones. Claude 3.5 Sonnet was the most accurate and consistent model overall, with the agent-based framework offering slight improvement. In contrast, GPT-4o Mini and Llama 3.1 8B struggled more, suggesting that larger model size and training quality significantly influence Text2SQL performance. Additionally, there was no clear difference in performance between agentless and agent-based frameworks for GPT-4o Mini and Llama 3.1 8B. Finally, GPT-4o Mini performs significantly better than Llama 3.1 8B, even though they are approximately the same size. This suggests that factors beyond model size, such as training data quality, fine-tuning strategies, and model architecture, play a crucial role in determining a model's ability to perform Text2SQL.

4.2 Execution Error Comparison

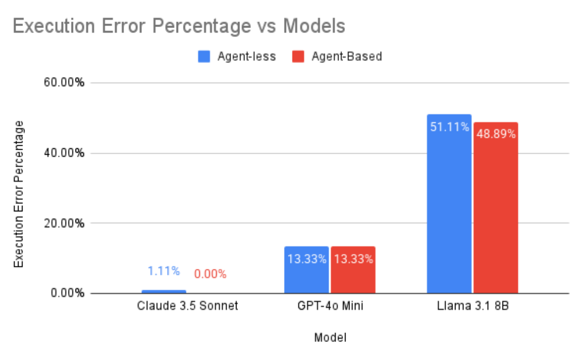


Figure 4: Execution Error Comparison

Execution error rate measures the proportion of generated queries that failed to run due to syntax errors, invalid table references, or other structural mistakes. Figure 4 summarizes the execution error rates across models and frameworks.

Claude 3.5 Sonnet was highly reliable, achieving an execution error rate of just 1.11% under the agentless framework and 0.00% under agent-based. This reflects highly reliable SQL generation. GPT-4o Mini was moderately reliable, with an execution error rate of 13.33% across both frameworks. Llama 3.1 8B had the highest execution error rates, exceeding 48% under both frameworks. This highlights significant challenges in generating syntactically correct SQL, even before considering semantic correctness.

Overall, larger and more capable models exhibited lower execution error rates, which aligns with trends observed in accuracy and consistency.

4.3 Agentless vs Agent-based Performance

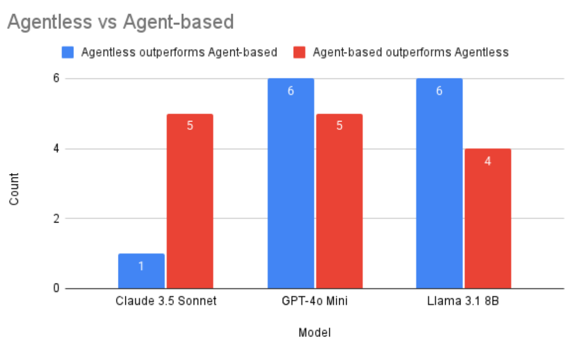


Figure 5: Agentless vs Agent-based Comparison

Finally, we compared the two frameworks by counting how often agentless or agent-based pipelines generated better results for a given query, summarized in Figure 5.

For Claude 3.5 Sonnet, the agent-based framework outperformed agentless in 5 queries, while agentless performed better in only 1 case. This suggests that Claude 3.5 Sonnet benefits from specialized agents that handle relevant table description extraction. For GPT-4o Mini, the agentless framework slightly outperformed agent-based (6 cases vs. 5 cases). Similarly, for Llama 3.1 8B, agentless was favored more often (6 cases vs. 4 cases). Thus, for GPT-4o Mini and Llama 3.1 8B, there isn't a clear significant advantage between using agentless vs agent-based frameworks.

Interestingly, Claude 3.5 Sonnet, being a larger model, performed better when paired with the agent-based framework. In contrast, GPT-4o Mini and Llama 3.1 8B, both smaller models, showed no clear significant advantage between agentless and agent-based pipelines. However, it cannot be definitively concluded that larger models benefit more from agent-based frameworks, because we only tested 1 large model (Claude 3.5 Sonnet). As a result, it is unclear whether the observed improvement is due to Claude's larger size or some Claude-specific characteristics, such as its training data, fine-tuning methods, or architectural design. To better understand the relationship between model size and framework effectiveness, future work would need to evaluate more large models to see if they exhibit similar performance differences between agentless and agent-based pipelines.

5 Conclusion

In this study, we conducted a differential analysis of three LLMs (Claude 3.5 Sonnet, GPT-4o Mini, and Llama 3.1 8B) across two different Text2SQL generation frameworks (agentless and agent-based). Our experiments on the BIRD-SQL Dev Set demonstrated that larger models like Claude 3.5 Sonnet are significantly more accurate, consistent, and robust compared to smaller models. Additionally, the agent-based framework offered slight improvements in performance Claude 3.5 Sonnet, possibly by mitigating issues such as the "Lost in the Middle" effect. However, for GPT-4o Mini and Llama 3.1 8B, which are smaller models, there was no clear advantage between the agent-based and agentless approaches. These findings highlight that both model choice and pipeline structure are important factors in designing effective Text2SQL systems, especially when targeting real-world applications that require reliability, scalability, and minimal user friction.

6 Future Directions

In our agent-based framework, we utilized a single Query Agent to generate one SQL query per NLQ. However, leading approaches such as CHASE-SQL and XiYan-SQL employ a multi-generator setup, producing multiple candidate queries followed by a selection phase to choose the best one. Incorporating multiple query generators (by potentially leveraging different LLMs) could help reduce hallucinations and improve overall accuracy by introducing diversity into the candidate pool. Thus, exploring a multi-generator and selection phase could enhance the reliability of generated SQL queries.

Our study included only one large-scale model (Claude 3.5 Sonnet) alongside two smaller models (GPT-4o Mini and Llama 3.1 8B). While Claude demonstrated improved performance under the agent-based framework, it cannot be definitively concluded that larger models inherently benefit more from agent-based designs. Further experimentation with a wider range of model sizes and architectures would help isolate whether the observed improvements are driven by model size or by Claude-specific characteristics.

Finally, while the agent-based framework may offer improvements in accuracy for certain models, it also incurs higher inference time and computational cost due to the additional LLM calls. Future work may include evaluating the trade-offs between performance gains and resource overheads. This is crucial for understanding the practical viability of deploying agent-based Text2SQL systems in real-world applications where efficiency and cost are important factors.

References

[1] A. B. Abacha, W. wai Yim, Y. Fu, Z. Sun, M. Yetisgen, F. Xia, and T. Lin. Medec: A benchmark for medical error detection and correction in clinical notes, 2025.

[2] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo, J. Gao, L. Mou, and Y. Li. A preview of xiyan-sql: A multi-generator ensemble framework for text-to-sql, 2025.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts, 2023.

[4] T. Mahmud, K. M. Azharul Hasan, M. Ahmed, and T. H. C. Chak. A rule based approach for nlp based query processing. In *2015 2nd International Conference on Electrical Information and Communication Technologies (EICT)*, pages 78–82, 2015.

[5] J. Murel and J. Noble. What is llm temperature?, 2024.

[6] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. O. Arik. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql, 2024.

[7] J. M. Zelle and R. J. Mooney. Learning to parse database queries using inductive logic programming. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence - Volume 2*, AAAI'96, pages 1050–1055. AAAI Press, 1996.